Update with Extra Data (Hashed Passwords) with FastAPI

In the previous chapter I explained to you how to update data in the database from input data coming from a **FastAPI** *path operation*.

Now I'll explain to you how to add **extra data**, additional to the input data, when updating or creating a model object.

This is particularly useful when you need to **generate some data** in your code that is **not coming from the client**, but you need to store it in the database. For example, to store a **hashed password**.

Password Hashing

Let's imagine that each hero in our system also has a **password**.

We should never store the password in plain text in the database, we should only stored a **hashed version** of it.

"**Hashing**" means converting some content (a password in this case) into a sequence of bytes (just a string) that looks like gibberish.

Whenever you pass exactly the same content (exactly the same password) you get exactly the same gibberish.

But you **cannot convert** from the gibberish **back to the password**.

Why use Password Hashing

If your database is stolen, the thief won't have your users' **plaintext passwords**, only the hashes.

So, the thief won't be able to try to use that password in another system (as many users use the same password everywhere, this would be dangerous).

> **Tip**
>
> You could use passlib to hash passwords.
>
> In this example we will use a fake hashing function to focus on the data changes. 🤡

Update Models with Extra Data

The `Hero` table model will now store a new field `hashed_password`.

And the data models for `HeroCreate` and `HeroUpdate` will also have a new field `password` that will contain the plain text password sent by clients.

Python 3.10+

```
# Code above omitted 👆

class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)
```

```python
class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    hashed_password: str = Field()


class HeroCreate(HeroBase):
    password: str


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    password: str | None = None

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from fastapi import FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    hashed_password: str = Field()


class HeroCreate(HeroBase):
    password: str


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    password: str | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def hash_password(password: str) -> str:
    # Use something like passlib here
    return f"not really hashed {password} hehehe"


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    hashed_password = hash_password(hero.password)
    with Session(engine) as session:
        extra_data = {"hashed_password": hashed_password}
        db_hero = Hero.model_validate(hero, update=extra_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(offset: int = 0, limit: int = Query(default=100, le=100)):
    with Session(engine) as session:
        heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
```

```python
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        extra_data = {}
        if "password" in hero_data:
            password = hero_data["password"]
            hashed_password = hash_password(password)
            extra_data["hashed_password"] = hashed_password
        db_hero.sqlmodel_update(hero_data, update=extra_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero
```

When a client is creating a new hero, they will send the `password` in the request body.

And when they are updating a hero, they could also send the `password` in the request body to update it.

Hash the Password

The app will receive the data from the client using the `HeroCreate` model.

This contains the `password` field with the plain text password, and we cannot use that one. So we need to generate a hash from it.

Python 3.10+

```python
# Code above omitted 👆

def hash_password(password: str) -> str:
    # Use something like passlib here
    return f"not really hashed {password} hehehe"

# Code here omitted 👈

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    hashed_password = hash_password(hero.password)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    hashed_password: str = Field()


class HeroCreate(HeroBase):
    password: str


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    password: str | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def hash_password(password: str) -> str:
    # Use something like passlib here
    return f"not really hashed {password} hehehe"


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    hashed_password = hash_password(hero.password)
    with Session(engine) as session:
        extra_data = {"hashed_password": hashed_password}
        db_hero = Hero.model_validate(hero, update=extra_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(offset: int = 0, limit: int = Query(default=100, le=100)):
    with Session(engine) as session:
        heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
```

```
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        extra_data = {}
        if "password" in hero_data:
            password = hero_data["password"]
            hashed_password = hash_password(password)
            extra_data["hashed_password"] = hashed_password
        db_hero.sqlmodel_update(hero_data, update=extra_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero
```

Create an Object with Extra Data

Now we need to create the database hero.

In previous examples, we have used something like:

```
db_hero = Hero.model_validate(hero)
```

This creates a `Hero` (which is a *table model*) object from the `HeroCreate` (which is a *data model*) object that we received in the request.

And this is all good... but as `Hero` doesn't have a field `password`, it won't be extracted from the object `HeroCreate` that has it.

`Hero` actually has a `hashed_password`, but we are not providing it. We need a way to provide it...

Dictionary Update

Let's pause for a second to check this, when working with dictionaries, there's a way to `update` a dictionary with extra data from another dictionary, something like this:

```
db_user_dict = {
    "name": "Deadpond",
    "secret_name": "Dive Wilson",
    "age": None,
}

hashed_password = "fakehashedpassword"

extra_data = {
    "hashed_password": hashed_password,
    "age": 32,
}

db_user_dict.update(extra_data)
```

```
print(db_user_dict)

# {
#     "name": "Deadpond",
#     "secret_name": "Dive Wilson",
#     "age": 32,
#     "hashed_password": "fakehashedpassword",
# }
```

This `update` method allows us to add and override things in the original dictionary with the data from another dictionary.

So now, `db_user_dict` has the updated `age` field with `32` instead of `None` and more importantly, **it has the new `hashed_password` field**.

Create a Model Object with Extra Data

Similar to how dictionaries have an `update` method, **SQLModel** models have a parameter `update` in `Hero.model_validate()` that takes a dictionary with extra data, or data that should take precedence:

Python 3.10+

```
# Code above omitted 👆

@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    hashed_password = hash_password(hero.password)
    with Session(engine) as session:
        extra_data = {"hashed_password": hashed_password}
        db_hero = Hero.model_validate(hero, update=extra_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    hashed_password: str = Field()


class HeroCreate(HeroBase):
    password: str


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    password: str | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def hash_password(password: str) -> str:
    # Use something like passlib here
    return f"not really hashed {password} hehehe"


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    hashed_password = hash_password(hero.password)
    with Session(engine) as session:
        extra_data = {"hashed_password": hashed_password}
        db_hero = Hero.model_validate(hero, update=extra_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(offset: int = 0, limit: int = Query(default=100, le=100)):
    with Session(engine) as session:
        heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
```

```
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        extra_data = {}
        if "password" in hero_data:
            password = hero_data["password"]
            hashed_password = hash_password(password)
            extra_data["hashed_password"] = hashed_password
        db_hero.sqlmodel_update(hero_data, update=extra_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero
```

Now, `db_hero` (which is a *table model* `Hero`) will extract its values from `hero` (which is a *data model* `HeroCreate`), and then it will **`update`** its values with the extra data from the dictionary `extra_data`.

It will only take the fields defined in `Hero`, so **it will not take the `password`** from `HeroCreate`. And it will also **take its values** from the **dictionary passed to the `update`** parameter, in this case, the `hashed_password`.

If there's a field in both `hero` and the `extra_data`, **the value from the `extra_data` passed to `update` will take precedence**.

Update with Extra Data

Now let's say we want to **update a hero** that already exists in the database.

The same way as before, to avoid removing existing data, we will use `exclude_unset=True` when calling `hero.model_dump()`, to get a dictionary with only the data sent by the client.

Python 3.10+

```
# Code above omitted 👆

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)

# Code below omitted 👇
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    hashed_password: str = Field()


class HeroCreate(HeroBase):
    password: str


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    password: str | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def hash_password(password: str) -> str:
    # Use something like passlib here
    return f"not really hashed {password} hehehe"


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    hashed_password = hash_password(hero.password)
    with Session(engine) as session:
        extra_data = {"hashed_password": hashed_password}
        db_hero = Hero.model_validate(hero, update=extra_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(offset: int = 0, limit: int = Query(default=100, le=100)):
    with Session(engine) as session:
        heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
```

```
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        extra_data = {}
        if "password" in hero_data:
            password = hero_data["password"]
            hashed_password = hash_password(password)
            extra_data["hashed_password"] = hashed_password
        db_hero.sqlmodel_update(hero_data, update=extra_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero
```

Now, this `hero_data` dictionary could contain a `password`. We need to check it, and if it's there, we need to generate the `hashed_password`.

Then we can put that `hashed_password` in a dictionary.

And then we can update the `db_hero` object using the method `db_hero.sqlmodel_update()`.

It takes a model object or dictionary with the data to update the object and also an **additional `update` argument** with extra data.

Python 3.10+

```
# Code above omitted 👆

@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        extra_data = {}
        if "password" in hero_data:
            password = hero_data["password"]
            hashed_password = hash_password(password)
            extra_data["hashed_password"] = hashed_password
        db_hero.sqlmodel_update(hero_data, update=extra_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero
```

👀 **Full file preview**

Python 3.10+

```python
from fastapi import FastAPI, HTTPException, Query
from sqlmodel import Field, Session, SQLModel, create_engine, select


class HeroBase(SQLModel):
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)


class Hero(HeroBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
    hashed_password: str = Field()


class HeroCreate(HeroBase):
    password: str


class HeroPublic(HeroBase):
    id: int


class HeroUpdate(SQLModel):
    name: str | None = None
    secret_name: str | None = None
    age: int | None = None
    password: str | None = None


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

connect_args = {"check_same_thread": False}
engine = create_engine(sqlite_url, echo=True, connect_args=connect_args)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def hash_password(password: str) -> str:
    # Use something like passlib here
    return f"not really hashed {password} hehehe"


app = FastAPI()


@app.on_event("startup")
def on_startup():
    create_db_and_tables()


@app.post("/heroes/", response_model=HeroPublic)
def create_hero(hero: HeroCreate):
    hashed_password = hash_password(hero.password)
    with Session(engine) as session:
        extra_data = {"hashed_password": hashed_password}
        db_hero = Hero.model_validate(hero, update=extra_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero


@app.get("/heroes/", response_model=list[HeroPublic])
def read_heroes(offset: int = 0, limit: int = Query(default=100, le=100)):
    with Session(engine) as session:
        heroes = session.exec(select(Hero).offset(offset).limit(limit)).all()
```

```python
        return heroes


@app.get("/heroes/{hero_id}", response_model=HeroPublic)
def read_hero(hero_id: int):
    with Session(engine) as session:
        hero = session.get(Hero, hero_id)
        if not hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        return hero


@app.patch("/heroes/{hero_id}", response_model=HeroPublic)
def update_hero(hero_id: int, hero: HeroUpdate):
    with Session(engine) as session:
        db_hero = session.get(Hero, hero_id)
        if not db_hero:
            raise HTTPException(status_code=404, detail="Hero not found")
        hero_data = hero.model_dump(exclude_unset=True)
        extra_data = {}
        if "password" in hero_data:
            password = hero_data["password"]
            hashed_password = hash_password(password)
            extra_data["hashed_password"] = hashed_password
        db_hero.sqlmodel_update(hero_data, update=extra_data)
        session.add(db_hero)
        session.commit()
        session.refresh(db_hero)
        return db_hero
```

Tip

The method `db_hero.sqlmodel_update()` was added in SQLModel 0.0.16. 😎

Recap

You can use the `update` parameter in `Hero.model_validate()` to provide extra data when creating a new object and `Hero.sqlmodel_update()` to provide extra data when updating an existing object. 🤓